

Text Classification for Spam Detection Using Supervised Machine Learning

Course: DLBAIPNLP01 - Project: NLP **Project type:** Text Classification for Spam Detection

Repository link: To be added after publication on GitHub or GitLab

Abstract

Spam messages are unsolicited messages that can expose users to phishing, fraud, and malware. This project develops a supervised natural language processing system that classifies short text messages as either legitimate or spam. The system uses the UCI SMS Spam Collection dataset, applies text normalization, stop-word removal, stemming, and feature encoding, and compares several machine learning pipelines. The evaluated models include Multinomial Naive Bayes, Linear Support Vector Classification, and Logistic Regression with Bag-of-Words and TF-IDF representations. The experiments are conducted progressively with subsets of 500, 1,500, 3,000, and 5,572 messages. On the full dataset, the best-performing configuration is Multinomial Naive Bayes with count-based features. It achieves an accuracy of 0.985, precision of 0.965, recall of 0.919, and F1-score of 0.942 on the held-out test set.

1. Introduction

Spam detection is a practical text classification problem in natural language processing. The objective of this project is to develop an NLP system that can distinguish legitimate messages from spam messages using supervised machine learning. The final product is a reproducible Python pipeline that downloads a public dataset, preprocesses messages, trains multiple classification models, evaluates them on held-out test data, and saves a trained model for later prediction.

The planned procedure follows the project task requirements. First, a labeled spam detection dataset is collected. Second, the text is preprocessed through normalization, stop-word removal, and stemming. Third, the text is encoded for use in machine learning models. Fourth, supervised classifiers are trained and evaluated. Finally, the trained model is used to classify new messages. The system is evaluated progressively, beginning with a small dataset and increasing the amount of available data until the complete dataset is used.

2. Theoretical Background and Related Work

Text classification is a supervised NLP task in which a model learns a relationship between text inputs and predefined labels. In spam detection, the input is a message and the label indicates whether the message is legitimate or spam. Before traditional machine learning models can process text, the text must be transformed into numerical features.

Two common feature extraction approaches are Bag-of-Words and Term Frequency-Inverse Document Frequency. Bag-of-Words represents documents by token counts. TF-IDF weights terms according to their frequency within a document and their rarity across the corpus. This project evaluates three

classical supervised models: Multinomial Naive Bayes, Linear Support Vector Classification, and Logistic Regression.

The evaluation uses accuracy, precision, recall, F1-score, and a confusion matrix. Accuracy alone is insufficient because the dataset is imbalanced. Precision measures how many predicted spam messages are actually spam. Recall measures how many real spam messages are detected. F1-score balances precision and recall. These metrics are important because false positives and false negatives have different practical consequences.

3. Project Planning and Resources

The project was planned as a sequence of practical phases: dataset acquisition, preprocessing and feature extraction, model training, evaluation across multiple dataset sizes, and documentation. The selected dataset was the UCI SMS Spam Collection because it is public, labeled, and directly relevant to the project task.

The software resources used were Python 3.12, pandas, scikit-learn, NLTK, matplotlib, and joblib. Pandas was used for dataset loading and summary statistics. Scikit-learn was used for vectorization, train-test splitting, model training, and evaluation. NLTK was used for stemming. Matplotlib was used to generate the confusion matrix plot. Joblib was used to save the trained model.

Several risks were considered. The first risk was class imbalance, since legitimate messages are far more common than spam messages. This was addressed with stratified train-test splitting and by reporting precision, recall, and F1-score. The second risk was overfitting on smaller samples, which was addressed through held-out test sets and progressive evaluation. The third risk was limited dataset scope, because the dataset contains SMS messages rather than all modern email or phishing formats.

4. Dataset and Preprocessing

The UCI SMS Spam Collection contains 5,572 labeled messages. Each message is labeled as `ham` or `spam`, where `ham` represents a legitimate message.

Class	Count	Percentage
Legitimate	4,825	86.59%
Spam	747	13.41%
Total	5,572	100.00%

The average message length is approximately 80.62 characters. The preprocessing pipeline lowercases text, replaces URLs with a marker, replaces numbers with a marker, removes punctuation and repeated whitespace, removes English stop words, and applies Snowball stemming. The processed tokens are encoded with CountVectorizer and TfidfVectorizer. Both vectorizers use unigrams and bigrams because short phrases can be informative in spam detection.

5. Model Development

The dataset was evaluated progressively with four dataset sizes: 500, 1,500, 3,000, and 5,572 messages. Each subset was sampled with stratification so that the spam-to-legitimate ratio remained representative. For every dataset size, an 80/20 train-test split was used. The random state was fixed to make the results reproducible.

Configuration	Vectorizer	Classifier
MultinomialNB + Count	Bag-of-Words	Multinomial Naive Bayes
MultinomialNB + TF-IDF	TF-IDF	Multinomial Naive Bayes
LinearSVC + TF-IDF	TF-IDF	Linear Support Vector Classification
Logistic Regression + TF-IDF	TF-IDF	Logistic Regression

The final saved model was selected from the full-dataset experiments because the full dataset provides the most defensible basis for a practical spam detection system.

6. Results and Evaluation

The strongest full-dataset result was achieved by Multinomial Naive Bayes with count-based features.

Dataset Size	Model	Vectorizer	Accuracy	Precision	Recall	F1-score
1,500	MultinomialNB	Count	0.990	0.974	0.950	0.962
5,572	MultinomialNB	Count	0.985	0.965	0.919	0.942
1,500	LinearSVC	TF-IDF	0.980	0.886	0.975	0.929
5,572	LinearSVC	TF-IDF	0.980	0.910	0.946	0.928
5,572	Logistic Regression	TF-IDF	0.962	0.806	0.946	0.870
5,572	MultinomialNB	TF-IDF	0.965	1.000	0.738	0.849

On the full held-out test set of 1,115 messages, the best model achieved 0.985 accuracy, 0.965 spam precision, 0.919 spam recall, and 0.942 spam F1-score. The TF-IDF Naive Bayes model had perfect precision on the full dataset, but its recall was much lower. LinearSVC achieved higher recall, but lower precision. The final model therefore represents a useful balance, especially because false positives can hide legitimate communication.

7. Reflection

The project objective was achieved. A complete NLP system was created that collects a labeled dataset, preprocesses text, trains supervised models, evaluates performance, and classifies new messages. The project also meets the requirement to evaluate different scenarios by testing multiple dataset sizes and multiple model configurations.

The strongest methodological feature is reproducibility. The dataset can be downloaded with one script, the experiments can be rerun with one command, and the outputs are written to a dedicated directory.

The main limitation is that the dataset is relatively small and SMS-specific. The system should not automatically be generalized to modern email spam, multilingual spam, or phishing campaigns with images and shortened links.

Future improvements could include larger and more recent datasets, cross-validation, hyperparameter tuning, threshold tuning, and comparison with transformer-based models. A small user interface or API could also be added to demonstrate practical integration.

8. Conclusion

This project developed a supervised NLP system for spam detection. The final pipeline uses the UCI SMS Spam Collection dataset, text normalization, stop-word removal, stemming, Bag-of-Words and TF-IDF encoding, and classical machine learning models. The best full-dataset result was achieved by Multinomial Naive Bayes with count-based features. It reached 0.985 accuracy and 0.942 spam F1-score on the held-out test set.

The results show that simple, well-structured NLP methods can be highly effective for spam detection when the data is clean and the evaluation process is carefully designed. Further work should focus on broader datasets, more recent spam patterns, threshold tuning, and comparison with neural language models.

References

Almeida, T. A., Hidalgo, J. M. G., & Yamakami, A. (2011). Contributions to the study of SMS spam filtering: new collection and results. *Proceedings of the 11th ACM Symposium on Document Engineering*.

Elakkiya, E., Selvakumar, S., & Leela Velusamy, R. (2021). TextSpamDetector: textual content based deep learning framework for social spam detection using conjoint attention mechanism. *Journal of Ambient Intelligence and Humanized Computing*, 12(10), 9287-9302.

Huan, H., Guo, Z., Cai, T., & He, Z. (2022). A text classification method based on a convolutional and bidirectional long short-term memory model. *Connection Science*, 34(1), 2108-2124.

Jurafsky, D., & Martin, J. (2013). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition* (2nd ed.). Pearson Prentice Hall.

UCI Machine Learning Repository. (n.d.). *SMS Spam Collection*. <https://archive.ics.uci.edu/dataset/228/sms+spam+collection>